1. Введение

Построение архитектуры проекта – важный этап в разработке информационной системы, который помогает определить структуру, компоненты, их взаимодействие и обеспечить устойчивость, масштабируемость и безопасность системы.

Шаблон проекта (архитектурный шаблон) – структурный идеальный образец, который позволяет стандартизировать архитектурные решения.

2. Процесс построения архитектуры проекта

1. Идентификация требований:

– Анализ требований к проекту

– Определение функциональных требований, бизнес-процессов и особенностей проекта

– Установка приоритетов и фокусировка на ключевых требованиях

2. Определение архитектурных принципов:

– Разработка архитектурных принципов, которые будут руководить построением системы

– Например, принципы безопасности, масштабируемости, гибкости и производительности

3. Выбор архитектурных паттернов:

– Выбор паттернов, соответствующих требованиям проекта

– Например, Model-View-Controller (MVC) для веб-приложений или микросервисная архитектура для распределённых систем

4. Определение компонентов и их взаимодействия:

– Разбить систему на компоненты

– Определить, как они будут взаимодействовать друг с другом

– Включает определение интерфейсов, API и данных, передаваемых между компонентами

5. Разработка архитектурных диаграмм:

– Создание диаграмм классов, диаграмм последовательности, диаграмм компонентов

– Визуализация структуры и взаимодействия компонентов

6. Учёт нефункциональных требований:

– Обеспечение выполнения нефункциональных требований (производительность, безопасность, масштабируемость)

– Через выбор соответствующих архитектурных решений и технологий

3. Популярные архитектурные шаблоны

3.1. Model-View-Controller (MVC)

Назначение: используется для построения веб-приложений.

Суть: разделяет систему на три компонента:

– Модель (Model) – хранение данных

– Представление (View) – отображение данных

– Контроллер (Controller) – обработка запросов и управление моделью и представлением

Компоненты:

Модель (Model):

– Отвечает за управление данными и бизнес-логикой приложения

– Не зависит от пользовательского интерфейса и представления данных

– Содержит информацию о состоянии приложения, бизнес-правила и методы для доступа к данным и их обновления

Представление (View):

– Отвечает за отображение данных пользователю и предоставляет пользовательский интерфейс

– Не содержит бизнес-логики и не обрабатывает данные, а только отображает их

– В веб-приложениях может быть HTML-страницей, шаблоном, виджетом и т. д.

– Получает данные из модели и отображает их пользователю

Контроллер (Controller):

– Является посредником между моделью и представлением

– Обрабатывает пользовательские запросы

– Взаимодействует с моделью для получения или обновления данных

– Управляет представлением для отображения результата

– Содержит логику обработки запросов, маршрутизацию и взаимодействие с моделью и представлением

Преимущества MVC:

– Разделение обязанностей – упрощает сопровождение и повторное использование кода

– Масштабируемость – каждый компонент может быть изменён или расширен независимо

– Тестирование – из-за чёткого разделения функциональности тестирование отдельных компонентов становится проще

– Гибкость и поддержка – при изменении требований к интерфейсу часто можно переиспользовать модель и контроллер, меняя только представление

Схема взаимодействия MVC:

```
text
Controller <--> Model
    |              ^
    | Send Data    | Request Information
    v              |
  View <-----------+
    |
    | Response
    v
Пользователь
```

3.2. Микросервисная архитектура (Microservices Architecture)

Назначение: используется для построения распределённых систем.

Суть: разделяет систему на небольшие, независимые микросервисы, каждый из которых отвечает за определённую функциональность.

Определение: структура разработки программного обеспечения, в которой приложение разбивается на небольшие, автономные и независимые сервисы, которые взаимодействуют друг с другом через API (Application Programming Interface).

Особенности:

– Каждый микросервис разрабатывается и поддерживается независимо

– Команды могут быстро вносить изменения в свой сервис без необходимости менять другие части приложения

– Микросервисы общаются между собой через сетевые вызовы по API

– Поскольку каждый микросервис независим, их можно разворачивать и обновлять отдельно

– Это позволяет ускорить процесс разработки и внедрения изменений

– Приложение может быть более устойчивым к отказам, так как отказ одного сервиса не обязательно приводит к отказу всего приложения

Требования для эффективного внедрения:

– Хорошее понимание принципов и лучших практик

– Правильный выбор инструментов для разработки, развертывания и управления сервисами

Пример микросервисной архитектуры для веб-приложения электронной коммерции:

– Каталог товаров

– Корзина покупок

– Аутентификация и управление пользователями

– Оформление заказа

– Сервис оплаты

Каждый из этих микросервисов может быть разработан, развернут и масштабирован независимо. Они взаимодействуют друг с другом через API, отправляя HTTP-запросы.

Сравнение Monolithic vs Microservices:

```
text
Monolithic Architecture:
- User Interface
- Business Logic
- Data Interface
- DB (одна база данных)
Microservices Architecture:
- User Interface
- Microservice (много)
- DB (у каждого своя)
```

3.3. Слоистая архитектура (Layered Architecture)

Назначение: разделяет систему на несколько уровней (слоёв), таких как представление, бизнес-логика и доступ к данным.

Определение: популярный подход к проектированию программных систем, в котором приложение разделяется на несколько логических слоёв или уровней. Каждый слой выполняет определённую функцию и взаимодействует только с ближайшими слоями, что способствует организации кода, облегчает его понимание и обеспечивает модульность.

Основные слои:

1. Представление (Presentation Layer):

– Отвечает за представление данных пользователю и управление пользовательским интерфейсом

– Содержит компоненты, связанные с отображением данных, обработкой пользовательского ввода и взаимодействием с пользователем

– Может включать веб-интерфейсы, графические интерфейсы пользователя (GUI) или API для взаимодействия с клиентскими приложениями

2. Бизнес-логика (Business Logic Layer):

– Содержит бизнес-логику приложения

– Происходит обработка данных, принятие бизнес-решений и взаимодействие с базой данных или другими источниками данных

– Управляет бизнес-процессами и бизнес-правилами, которые определяют, как приложение должно работать

3. Слой доступа к данным (Data Access Layer):

– Отвечает за доступ к данным и взаимодействие с хранилищами данных (базы данных, файлы, внешние сервисы)

– Обеспечивает абстракцию от конкретных источников данных и предоставляет бизнес-логике доступ к данным

4. Инфраструктурный слой (Infrastructure Layer):

– Содержит общие компоненты и утилиты, необходимые для функционирования приложения

– Система управления конфигурацией, система логирования, аутентификация, кэширование и другие инфраструктурные аспекты

Преимущества слоистой архитектуры:

– Модульность – каждый слой является независимым модулем, что упрощает разработку, тестирование и обслуживание

– Чистый дизайн – разделение приложения на слои помогает соблюдать принципы чистой архитектуры и отделить бизнес-логику от деталей реализации

– Повторное использование – слои могут быть повторно использованы в разных частях приложения или в разных проектах

– Масштабируемость – каждый слой может быть масштабирован независимо, что облегчает оптимизацию производительности

– Понимание и обслуживание – чёткая организация слоёв делает код более понятным и облегчает обслуживание и доработку приложения

Пример работы:
Когда пользователь открывает веб-интерфейс и делает запрос на просмотр каталога товаров, запрос сначала попадает в представление. Представление передаёт запрос на бизнес-логику, которая обрабатывает запрос, консультируясь с данными из слоя доступа к данным. Бизнес-логика получает необходимые данные о товарах из базы данных, выполняет расчёты и формирует ответ, который отправляется обратно в представление для отображения пользователю.

3.4. Архитектура «Чистая архитектура» (Clean Architecture)

Определение: архитектурный подход, разработанный Робертом Мартином. Призван упростить проектирование, разработку и сопровождение программных систем, делая их более модульными, гибкими и тестируемыми.

Цель: изолировать бизнес-логику от зависимостей от фреймворков, библиотек и инфраструктуры.

Принципы «Чистой архитектуры»:

1. Разделение ответственности (Separation of Concerns):

Система разделяется на различные уровни или слои, каждый из которых отвечает за определённую ответственность:

– Слой представления (Presentation Layer) – пользовательский интерфейс и взаимодействие с пользователем

– Слой бизнес-логики (Business Logic Layer) – бизнес-правила и бизнес-логика приложения

– Слой данных (Data Layer) – доступ к данным (базы данных или внешние API)

2. Зависимость наружу (Dependency Inversion):

– Высокоуровневые модули не должны зависеть от низкоуровневых модулей

– Оба должны зависеть от абстракций

– Способствует уменьшению связности между компонентами и увеличивает гибкость системы

3. Граничные объекты (Boundary Objects):

– Служат связующим звеном между внешним миром (пользовательским интерфейсом или базой данных) и внутренней бизнес-логикой

– Позволяют изолировать внутреннюю логику от конкретных технологий и упрощают тестирование

4. Использование презентационного шаблона (Presenter):

– Для отделения бизнес-логики от пользовательского интерфейса часто используется презентационный шаблон

– Например, MVP (Model-View-Presenter) или MVVM (Model-View-ViewModel)

– Это позволяет разработчикам создавать интерфейс, который независим от логики и бизнес-правил

5. Соблюдение принципа единственной ответственности (Single Responsibility Principle):

– Каждый модуль, класс или компонент должен иметь только одну причину для изменения

– Помогает уменьшить сложность и повысить поддерживаемость системы

Результат: архитектура призвана сделать систему более гибкой, легко расширяемой и поддерживаемой, а также обеспечить её независимость от конкретных технологий.

3.5. Событийно-ориентированная архитектура (Event-Driven Architecture, EDA)

Определение: архитектурный подход, который ориентирован на обработку и передачу событий между компонентами системы. Система строится вокруг генерации, передачи и обработки событий, что позволяет создавать более гибкие, масштабируемые и реактивные приложения.

Основные концепции и компоненты:

1. События (Events):

– Сигналы или уведомления о произошедших действиях или изменениях в системе

– Могут быть сгенерированы различными компонентами (пользовательский интерфейс, сервисы, базы данных и т. д.)

– Содержат информацию о событии (тип события и данные, связанные с ним)

– Могут быть асинхронно переданы другим компонентам

2. Источники событий (Event Sources):

– Компоненты или системы, которые генерируют события

– Примеры: веб-приложения, базы данных и другие приложения

3. Брокеры событий (Event Brokers):

– Компоненты, которые принимают события от источников и маршрутизируют их к соответствующим обработчикам событий

– Выполняют функцию посредника и позволяют разным компонентам системы общаться между собой, не зная друг о друге напрямую

– Могут быть централизованными (система сообщений) или децентрализованными (публикация-подписка)

4. Обработчики событий (Event Handlers):

– Компоненты или функции, которые реагируют на события и выполняют соответствующие действия

– Могут подписываться на определённые типы событий и реагировать на них

– Могут выполнять различные действия: обновление базы данных, отправка уведомлений, запуск вычислительных процессов и т. д.

Преимущества событийно-ориентированной архитектуры:

– Гибкость и реактивность – EDA обеспечивает гибкую и реактивную модель разработки, позволяя системе мгновенно реагировать на события и изменения

– Масштабируемость – позволяет легко масштабировать систему, добавляя новые источники, брокеры и обработчики событий

– Отделение компонентов – EDA способствует отделению компонентов, позволяя им взаимодействовать через события, что делает систему более модульной и поддерживаемой

4. Сравнительная таблица архитектурных шаблонов

Шаблон	Назначение	Ключевая идея	Преимущества
MVC	Веб-приложения	Разделение на Model, View, Controller	Разделение обязанностей, масштабируемость, тестируемость
Микросервисы	Распределённые системы	Разбиение на независимые сервисы	Независимость разработки, устойчивость к отказам, гибкость
Слоистая	Общее проектирование	Разделение на слои (представление, бизнес-логика, данные, инфраструктура)	Модульность, чистый дизайн, повторное использование
Чистая архитектура	Общее проектирование	Изоляция бизнес-логики от фреймворков	Модульность, гибкость, тестируемость, независимость от технологий
Событийно-ориентированная	Реактивные системы	Обмен событиями между компонентами	Гибкость, реактивность, масштабируемость, отделение компонентов